Mocking in Apex

Why It Changes Everything

FFLib Series · Session 005

John Storey

Staff Software Engineer · Thrivent



About me

Staff Software Engineer | Thrivent | Apex Enterprise Patterns maintainer

- **Apex Enterprise Patterns** — a maintainer of fflib, the library this series is built on
- **FinancialForce roots** — helped build fflib in the mid-2010s, alongside Andy Fawcett, John Daniel ("JD") and others

When I'm not working in the IT cloud, I'm flying above the real ones as a general aviation pilot.

thrivent®



FFLib Series

Promises we made

Andy, Session 1: `newInstance()`, not `new`, "for reasons we'll talk about in the mocking sessions"	→ slide **13**
Andy, Session 1: instance methods and top-level classes mock; statics don't	→ slide **12**
Andy, Session 1: the Application class is optional	→ slide **16**
Andy, Session 2: mocking the Unit of Work, skipped "for another session"	→ slide **23–24**
John Daniel, Session 3: the Domain's static mock, "the hook… in the fifth episode"	→ slide **14**
John Storey, Session 4: can mocking handle statics? And why `virtual`?	→ slide **12, 15**

The Widgets example

Small enough to fit on a slide

- `WDGT_Widget__c` — an auto-number Name, plus `WDGT_Guid__c` and `WDGT_MightBeRequired__c`
- **One Service method, three collaborators** — a Selector, a GUID generator, and a Unit of Work
- **The `WDGT_` prefix** — a namespace inside one package; remember it for Session 6

WDGT_WidgetsService.cls

```
1  public virtual void tagWidgetsWithGuid(Set<Id> widgetsIdSet) {
2    if (widgetsIdSet == null || widgetsIdSet.isEmpty()) {
3      return;
4    }
5    try {
6      fflib_SObjectUnitOfWork uow = WDGT_UnitOfWork.newInstance();                  // Unit of Work
7      for (WDGT_Widget__c widget : widgetSelector.selectById(widgetsIdSet)) {  // Selector
8        widget.WDGT_Guid__c = widgetsUtility.generateGuid();                       // GUID generator
9        uow.registerDirty(new WDGT_Widget__c(Id = widget.Id, WDGT_Guid__c = widget.WDGT_Guid__c));
10     }
11     uow.commitWork();
12   } catch (Exception e) {
13     throw wrap(e);
14   }
15 }
```

How should we test it?

A test that uses the database

What's wrong with this test?

WDGT_UnitOfWorkTest.cls

```
@IsTest
private static void tagWidgetsWithGuid_WHEN_twoWidgetsSaved_EXPECT_eachSavedWithUniqueGuid() {
  User widgetUser = newWidgetUser();     // query a profile, insert a user, assign a permission set
  System.runAs(widgetUser) {
    List<WDGT_Widget__c> widgets = new List<WDGT_Widget__c>{
      new WDGT_Widget__c(), new WDGT_Widget__c()
    };
    insert widgets;
    Set<Id> widgetIds = new Map<Id, WDGT_Widget__c>(widgets).keySet();

    WDGT_WidgetsService.newInstance().tagWidgetsWithGuid(widgetIds);

    List<WDGT_Widget__c> saved = [
      SELECT Id, WDGT_Guid__c FROM WDGT_Widget__c WHERE Id IN :widgetIds
    ];
    Assert.areEqual(32, saved[0].WDGT_Guid__c?.length(), '…');
    Assert.areEqual(32, saved[1].WDGT_Guid__c?.length(), '…');
    Assert.areNotEqual(saved[0].WDGT_Guid__c, saved[1].WDGT_Guid__c, '…');
  }
}
```

✗ **Setup is about the org, not the method** — a user, a permission set, saved records: none of it is tagging

✗ **Can't check the GUIDs** — they're random: only "32 characters, and different"

✗ **Tests `generateGuid()` too** — logic calling logic

✗ **Slow** — ≈2–2.7 s, 4 SOQL, 6 DML; mocked: ≈10–35 ms, none

✗ **Fragile** — an unrelated validation rule breaks it

✗ **Proves the database works** — Salesforce's job, not ours

💡 Coverage isn't quality.

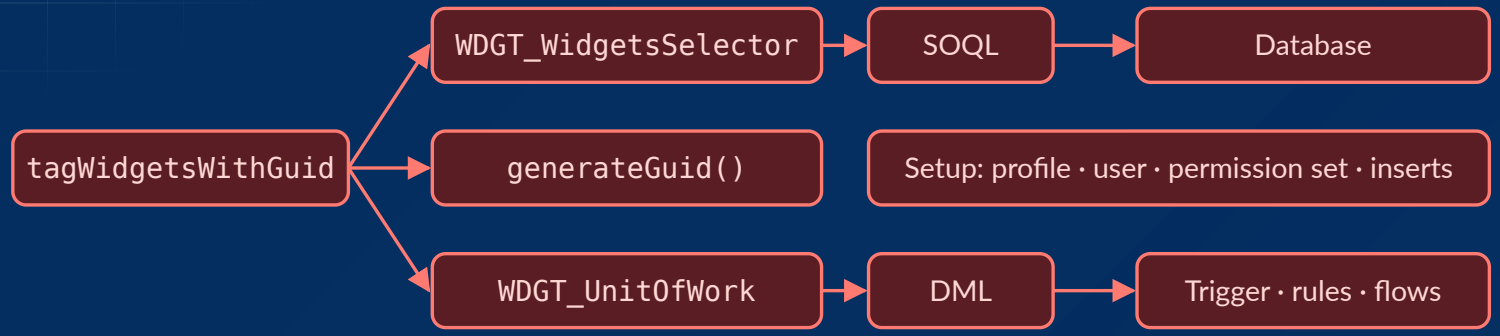
What a unit test should touch

Test only the method in front of you



Unit band: Andy, Session 1

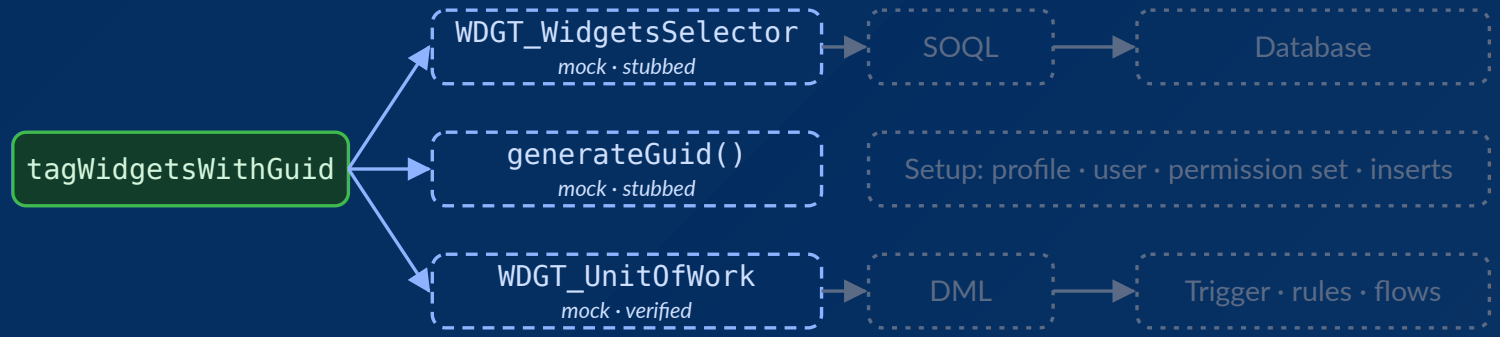
❌ The DML test — an integration test pretending to be a unit test



GOVERNOR LIMITS USED

SOQL queries	4 / 100
DML statements	6 / 150
Run time	≈2–2.7 s

✅ The mocked test — only the method runs; everything it calls is a mock



GOVERNOR LIMITS USED

SOQL queries	0 / 100
DML statements	0 / 150
Run time	≈10–35 ms

...and a Widget is a tiny object with no automation. Every trigger, flow and related record grows the top row.

Now imagine your own, more complex logic...

Testing through the database, at scale

The long way to a one-line method

An illustrative test, not from the repo: to test `Invoice.isOverdue()`, the test first walks a real record through its whole life.

1 Draft

insert Account, Contact, Product, Pricebook, Order

- AccountTrigger
- ContactTrigger
- OrderTrigger
- OrdersService
- AccountsSelector
- Territory flow
- ProductsSelector
- Duplicate rule

SOQL 14/100
DML 9/150
CPU 1.1/10 s
Heap 0.6/6 MB

2 Submitted

update Status

- OrdersHandler
- Orders.validate
- CreditCheckService
- Approval flow
- UsersSelector
- Sharing recalc
- OrdersSelector

SOQL 27/100
DML 22/150
CPU 2.4/10 s
Heap 1.3/6 MB

3 Approved

approve, notify

- ApprovalService
- NotificationService
- TaskTrigger
- QueuesSelect
- Escalation flow
- EmailService
- Validation rules ×4
- + Commissions
- + Partner sync

Another team, last sprint:
+12 SOQL, +9 DML

SOQL 41/100
DML 46/150
CPU 3.9/10 s
Heap 2.2/6 MB

4 Fulfilled

reserve stock, book a carrier

- InventoryService



CPU 5.3/10 s
Heap 3.1/6 MB

5 Shipped

tracking, customer case

CPU 6.6/10 s
Heap 4.0/6 MB

6 Invoiced

invoice, lines, tax, roll-ups

- InvoicingService
- InvoiceTrigger
- InvoiceLineTrigger
- TaxService
- Roll-up flow
- InvoicesSelector
- OrderTrigger ↺

SOQL 89/100

✖ Test failed

```
System.
LimitException:
Too many SOQL
queries: 101
```

7 Finally: the test

Never reached

Nothing changed here, or in Invoiced.
The change was in Approved.

Mocked: one `Invoice__c` in memory. **0 SOQL, 0 DML, milliseconds**, and no other team's change can break it.

Stubs, mocks, and fakes

Three kinds of stand-in

- **Stub** — canned method returns: `when(...).thenReturn(...)`
- **Mock** — a stand-in class that records its calls for the test to verify, with Mockito-style methods: `verify`, matchers

> One ApexMocks object does both: it stubs answers and records calls.

- **Fake** — a working stand-in: `WDGT_InertDml` saves nothing, but remembers what it would have saved

Tests that exist for coverage

Three special cases

Coverage isn't quality, but some things just need coverage: the 0.01%.

Case	Why it's special	How we cover it
Selectors	All about SOQL	Call each select method with empty parameters `WDGT_WidgetsSelectorTest`
Unit of Work	All about DML	Inject an inert IDML, register some things, and commit `WDGT_InertDml`
Triggers	Only run because of DML; must have more than 0% coverage to deploy	Make the trigger handler inert, then save something `WDGT_WidgetsTriggerHandlerTest` ⚠️ Beware validation rules, flows, and other automation

The Stub API

The Stub API does the work; ApexMocks wraps it in Mockito-style stubbing and verifying (next slide).

A hand-written stub (illustrative)

```
1  private class GuidStub implements System.StubProvider {
2    public Object handleMethodCall(Object stubbedObject, String stubbedMethodName, Type returnType,
3        List<Type> listOfParamTypes, List<String> listOfParamNames, List<Object> listOfArgs) {
4      return stubbedMethodName == 'generateGuid' ? '11111111111111111111111111111111' : null;
5    }
6  }
7
8  WDGT_WidgetsUtility utility =
9    (WDGT_WidgetsUtility) Test.createStub(WDGT_WidgetsUtility.class, new GuidStub());
10 utility.generateGuid();   // '11111111111111111111111111111111'
```

- **A platform feature, tests only** (GA in Spring '17): `Test.createStub(Type, System.StubProvider)` returns an object of that type
- **One hook:** every call on it lands in your provider's `handleMethodCall`; whatever that returns, the caller gets
- **Nothing else built in:** no `when`, no `verify`, no matchers

How ApexMocks works

One mock, three phases

Inside fflib_ApexMocks.cls

```apex
public Object mock(Type classToMock) {
  return Test.createStub(classToMock, this);
}
```

In a test (from WDGT_UnitOfWorkTest, trimmed)

```apex
// Creating the mock
fflib_ApexMocks mocks = new fflib_ApexMocks();
WDGT_WidgetsUtility utilityMock =
  (WDGT_WidgetsUtility) mocks.mock(WDGT_WidgetsUtility.class);

// Given: you stub
mocks.startStubbing();
mocks.when(selectorMock.selectById(widgetIds))
  .thenReturn(new List<WDGT_Widget__c>{ first });
mocks.when(utilityMock.generateGuid()).thenReturn(firstGuid);
mocks.stopStubbing();

// When: your code runs
new WDGT_WidgetsService(selectorMock, utilityMock)
  .tagWidgetsWithGuid(widgetIds);

// Then: you verify
((WDGT_WidgetsUtility) mocks.verify(utilityMock)).generateGuid();
Assert.areEqual(firstGuid, first.WDGT_Guid__c,
  'The widget should get the stubbed GUID');
```

Your test, as ApexMocks sees it

Creating the mock: `mocks.mock(X.class)` asks the Stub API for a stand-in `X`. From now on, every call on it goes to ApexMocks.

Given: you stub (between `startStubbing()` and `stopStubbing()`). Calls on the mock aren't real here: they tell ApexMocks what to answer later. `when(mock.method(args)).thenReturn(value)`

When: your code runs. Each call on the mock is remembered, and gets back the answer you stubbed, or `null` if there isn't one.

Then: you verify. After `verify(mock)`, the next call is a check, not a real call: "was this called, with these arguments?"

Answers and checks match the method and its exact arguments. Matchers loosen that (slide 19).

What the Stub API can't stub

The limits that shape our code

It can't stub…	So we write…
Static methods, including `@future`	Instance methods: Services are instances, created with `newInstance()`
Inner classes	Top-level classes for anything we'll mock
A class with only private constructors	A public constructor; `newInstance()` calls it
Private methods, properties	Public methods for collaborators; a mock never runs its constructor (slide 25)
System types, triggers, `Batchable` classes	Our own wrapper, mocked instead: `generateGuid()` wraps `UUID.randomUUID()`
Classes in another namespace	Mocks created in the class's own namespace

✅ Andy, Session 1: instance methods, top-level classes · ✅ Session 4: statics? No.

Injecting collaborators

One way in, one way to customize

WDGT_WidgetsService.cls

```
1  public static WDGT_WidgetsService newInstance() {              // one way in: live collaborators
2    return new WDGT_WidgetsService(
3      WDGT_WidgetsSelector.newInstance(),
4      WDGT_WidgetsUtility.newInstance()
5    );
6  }
7
8  public WDGT_WidgetsService(WDGT_WidgetsSelector widgetSelector,   // one way to customize
9                             WDGT_WidgetsUtility widgetsUtility) {
10   this.widgetSelector = widgetSelector;
11   this.widgetsUtility = widgetsUtility;
12 }
```

Callers

```
1 // app code
2 WDGT_WidgetsService.newInstance().tagWidgetsWithGuid(widgetIds);
3 // unit test
4 new WDGT_WidgetsService(selectorMock, utilityMock).tagWidgetsWithGuid(widgetIds);
```

💡 Inject mocks through the constructor, so a test has no hidden setup.

✅ Andy, Session 1: `newInstance()`, not `new`

Substituting stateful objects

Swap them where they're made

- **Some objects carry state.** A Domain wraps the records it was given; a Unit of Work collects changes until it commits.
- **They hold state while in use,** so each piece of work needs a fresh one; a constructor shouldn't hand in one to be reused.
- **So we swap them where they're made:** their `newInstance()` returns the test's mock when one is set.

Unit of Work: `WDGT_UnitOfWork.cls`

```
1 @TestVisible
2 private static fflib_SObjectUnitOfWork mock;
3
4 public static fflib_SObjectUnitOfWork newInstance() {
5   if (mock != null) {
6     return mock;
7   }
8   return new fflib_SObjectUnitOfWork(
9     new List<SObjectType>{ WDGT_Widget__c.SObjectType },
10    new fflib_SObjectUnitOfWork.UserModeDML()
11  );
12 }
13
14 // In a test
15 WDGT_UnitOfWork.mock = uowMock;
```

Domain, from the Warehouse app: `Robots.cls`

```
1 @TestVisible
2 private static Robots mock;
3
4 public static Robots newInstance(List<Robot__c> recordList) {
5   return mock != null ? mock : new Robots(recordList);
6 }
7
8 // In a test
9 Robots.mock = robotsMock;
```

> *"Domains and UnitOfWork still use property injection – as they are stateful (domain within a method, uow's within a service method)"*
>
> Andy Fawcett, sample PR #61, Sept 2026

✅ John Daniel, Session 3: "the hook... in the fifth episode"

Making a class mockable

A short checklist

- ☑ **A non-private constructor**
- ☑ **A top-level class**
- ☑ **Instance methods** for anything you'll stub
- ☑ **Nothing private** you need to stub
- ✕ `virtual` **: not needed for Apex mocking**, unlike some other stacks. Use it only when you need inheritance

✅ Session 4: "maybe we could ping Andy about why he likes virtual"

Where fflib is heading

Mainstream dependency injection

- **A static factory plus constructor injection** — `newInstance()` by default, constructors for customized instances: the same shape as Spring, .NET, or any Mockito-era stack
- **Interfaces only where implementations genuinely vary** — the refreshed sample keeps just `ISupportInvoicing`; the Widgets code has none
- **The Application class is optional**
- **AT4DX keeps a factory, on purpose** — across packages, the caller can't name the class that does the work: another package supplies it through custom metadata, maybe installed later or overriding another. So the caller asks a factory for an interface, resolved at run time; tests still `setMock()` it. More in Session 6

> *"Having an Application class is not a requirement to use FFLib; it depends on your needs, particularly regarding dependency injection."*
>
> Andy Fawcett, andyinthecloud.com, April 2026

✅ Andy, Session 1: the Application class is optional

How fflib mocking evolved

Not wrong, just earlier

1

Generated mocks

ApexMocks generated mock classes, for interfaces only. So interfaces went everywhere

2

The Application factory

Interfaces plus a central factory, and `setMock` to swap in the generated mocks

3

The Stub API arrives

Mocks made at run time, for classes too; the generator, and the need for interfaces everywhere, fade away

4

Today

Concrete classes, `newInstance()` plus constructors, interfaces only where they vary

5

Next: multi-package

AT4DX: the same seam, with metadata deciding (Session 6)

> Interfaces everywhere and an Application factory? Not wrong; just from an earlier stage.

Stubbing and verifying

The three you'll use daily

- ① `thenReturn` — what a call answers
- ② `thenReturnMulti` — one answer per call, in order
- ③ `verify` — the call happened, with these arguments

WDGT_UnitOfWorkTest.cls — the mocked test

```
// Given
fflib_ApexMocks mocks = new fflib_ApexMocks();
WDGT_WidgetsSelector selectorMock = (WDGT_WidgetsSelector) mocks.mock(WDGT_WidgetsSelector.class);
WDGT_WidgetsUtility utilityMock = (WDGT_WidgetsUtility) mocks.mock(WDGT_WidgetsUtility.class);
fflib_SObjectUnitOfWork uowMock = (fflib_SObjectUnitOfWork) mocks.mock(fflib_SObjectUnitOfWork.class);
mocks.startStubbing();
mocks.when(selectorMock.selectById(widgetIds)).thenReturn(new List<WDGT_Widget__c>{ first, second }); // ①
mocks.when(utilityMock.generateGuid()).thenReturnMulti(new List<Object>{ firstGuid, secondGuid });     // ②
mocks.stopStubbing();
WDGT_UnitOfWork.mock = uowMock;

// When
new WDGT_WidgetsService(selectorMock, utilityMock).tagWidgetsWithGuid(widgetIds);

// Then
((fflib_SObjectUnitOfWork) mocks.verify(uowMock))                                                    // ③
  .registerDirty(new WDGT_Widget__c(Id = first.Id, WDGT_Guid__c = firstGuid));
((fflib_SObjectUnitOfWork) mocks.verify(uowMock))
  .registerDirty(new WDGT_Widget__c(Id = second.Id, WDGT_Guid__c = secondGuid));
((fflib_SObjectUnitOfWork) mocks.verify(uowMock, 1)).commitWork();
```

💡 Exact first: SObject equality proves the right GUID *and* nothing beyond Id and GUID.

Other ways to verify

More than one way to bake a cake



No cats were harmed in the making of these tests.

Slide 18's **Then**, three more ways. Same Given and When; real tests in `WDGT_UnitOfWorkTest`.

Matchers and a count

when you can't be exact

```
((fflib_SObjectUnitOfWork)
  mocks.verify(uowMock, 2))
  .registerDirty(
    fflib_Match.anySObject());

((fflib_SObjectUnitOfWork)
  mocks.verify(uowMock))
  .registerDirty(
    fflib_Match.sObjectWith(
      new Map<SObjectField, Object>{
        WDGT_Widget__c.Id => first.Id,
        WDGT_Widget__c.WDGT_Guid__c
          => firstGuid
      }));
```

Argument captor

when you want to look at what was passed

```
fflib_ArgumentCaptor captor =
  fflib_ArgumentCaptor.forClass(
    WDGT_Widget__c.class);

((fflib_SObjectUnitOfWork)
  mocks.verify(uowMock, 2))
  .registerDirty(
    (SObject) captor.capture());

List<Object> registered =
  captor.getAllValues();
Assert.areEqual(firstGuid,
  ((WDGT_Widget__c) registered[0])
    .WDGT_Guid__c, '…');
```

In order

when order is the logic

```
fflib_InOrder inOrder =
  mocks.inOrder(
    new List<Object>{ uowMock });

((fflib_SObjectUnitOfWork)
  inOrder.verify(uowMock,
    mocks.calls(2)))
  .registerDirty(
    fflib_Match.anySObject());

((fflib_SObjectUnitOfWork)
  inOrder.verify(uowMock,
    mocks.calls(1)))
  .commitWork();
```

A matcher returns a typed placeholder (`null`) and waits on the side, so generic ones need a cast, and a call's arguments are all matchers or none.

ApexMocks reference card

If you've wished for it, it's probably here

Stubbing

- `when(…).thenReturn(v)`
- `thenReturnMulti(list)` — consecutive calls
- `thenThrow(e)`, `thenThrowMulti(list)`
- `thenAnswer(fflib_Answer)` — computed from the arguments
- Void methods: `doThrowWhen(e, mock)`, `doAnswer(answer, mock)`

Verifying

- `verify(mock)` = once; `verify(mock, n)`; `fflib_ApexMocks.NEVER`
- `times(n)`, `never()`, `atLeast(n)`, `atMost(n)`, `atLeastOnce()`, `between(a, b)`
- `.description('…')` — your own failure message
- `inOrder(…)` with `calls(n)`, `verifyNoMoreInteractions()`, `verifyNoInteractions()`
- `fflib_ArgumentCaptor`: `capture()`, `getValue()`, `getAllValues()`

Matchers (`fflib_Match`; all or nothing per call)

- **Any:** `anyString`, `anyId`, `anySObject`, `anyObject`, …
- **Equality:** `eq`, `eqId`, `refEq`, `isNull`, `isNotNull`
- **SObjects:** `sObjectWith(fieldMap)`, `sObjectWithId`, `sObjectWithName`, `sObjectOfType`, `sObjectsWith`
- **Strings, lists:** `stringContains`, `stringStartsWith`, `stringMatches`, `listContains`, `listIsEmpty`
- **Ranges, dates:** `integerBetween`, `decimalMoreThan`, `dateAfter`, `datetimeBetween`
- **Combining:** `allOf`, `anyOf`, `noneOf`, `isNot`
- **Your own:** implement `fflib_IMatcher`

Further reading

- Mockito's method reference: javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html
- ApexMocks: github.com/apex-enterprise-patterns/fflib-apex-mocks (README, "Documentation")

Building records in memory

No inserts required

Test setup, no DML (snippets)

```
// A fake Id: valid-looking, no insert
WDGT_Widget__c widget = new WDGT_Widget__c(
  Id = fflib_IDGenerator.generate(WDGT_Widget__c.SObjectType)
);

// Read-only fields (auto-number Name, formulas, roll-ups): set them through JSON
widget = (WDGT_Widget__c) fflib_ApexMocksUtils.setReadOnlyFields(
  widget, WDGT_Widget__c.class,
  new Map<SObjectField, Object>{ WDGT_Widget__c.Name => 'Widget_000000001' }
);

// A parent: set the relationship field directly (as the Warehouse tests do)
line.FulfillmentOrder__r = order;

// Children: makeRelationship builds parents with their child lists
List<Account> accounts = (List<Account>) fflib_ApexMocksUtils.makeRelationship(
  List<Account>.class, new List<Account>{ account },
  Contact.AccountId, new List<List<Contact>>{ contacts }
);
```

💡 Build records in memory, not in the database. Never `SeeAllData`.

Testing Selectors

Mostly coverage, never pointless

- **Callers mock them, so the quality lives in the callers' tests.** The Selector's own test is mostly for coverage (slide 9): empty conditions, no data, no DML. As a bonus, it proves the SOQL is valid.
- **No early return.** Convert null to empty and let the query run. Not intuitive, but a guard skips the query: no coverage, unchecked SOQL. It costs one query of your 100, and about 2 ms. Null *must* be converted: it throws in fflib's dynamic SOQL.

The Selector and its test

```apex
public virtual List<WDGT_Widget__c> selectById(Set<Id> idSet) {
  idSet = (idSet != null) ? idSet : new Set<Id>();
  return (List<WDGT_Widget__c>) selectSObjectsById(idSet);
}

// WDGT_WidgetsSelectorTest
Integer fromEmpty = selector.selectById(new Set<Id>()).size();
Integer fromNull = selector.selectById(null).size();
Assert.areEqual(0, fromEmpty, 'Empty Ids: no widgets');
Assert.areEqual(0, fromNull, 'Null Ids: no widgets');
```

⚠️ What a mocked Selector hides

```apex
// The Selector doesn't query WDGT_MightBeRequired__c.
// A real row throws when logic reads it:
//   SObjectException: SObject row was retrieved via
//   SOQL without querying the requested field
String fromReal = selected.WDGT_MightBeRequired__c;

// A mocked Selector returns what the test built,
// so the same read works ('Present') and the gap hides:
String fromMock =
  selectorMock.selectById(ids)[0].WDGT_MightBeRequired__c;
```

💡 Test our logic, not Salesforce's. E2E test automation catches the field gap.

Testing with Unit of Work

Calls or outcome?

If...	Use	Assert with
You don't care about the resulting records	A fully mocked Unit of Work (ApexMocks)	`verify(uowMock).registerDirty(…)`, `commitWork()`, or capture the records (slide 19)
You want to inspect what would have been saved	A real Unit of Work with an inert IDML (`WDGT_InertDml`, next slide)	`Assert` on `inertDml.inserted` / `updated` / `deleted`

Mocked Unit of Work + verify or captor

sees the calls

Asks "Which `register*` method, with which records?"

Tied to The exact method and overload

Breaks when `registerDirty` → `registerUpsert`, or one record → a list, *even with the same result*

Real Unit of Work + inert IDML

sees the outcome

Asks "What would have been saved?"

Tied to Nothing about how it was registered

Breaks when Only when the result changes. Also runs the real commit logic

✅ Andy, Session 2: mocking the Unit of Work, "for another session"

Inspecting what would be saved

InertDml: a database that saves nothing

- **The real Unit of Work runs its full commit logic,** but the IDML *collects* the records instead of saving them.

`WDGT_InertDml.cls` **(trimmed)**

```apex
@IsTest
public class WDGT_InertDml implements
    fflib_SObjectUnitOfWork.IDML,
    fflib_SObjectUnitOfWork.IDMLUpsertable {
  public List<SObject> inserted = new List<SObject>();
  public List<SObject> updated = new List<SObject>();
  // … upserted, deleted, published, emptied

  public void dmlInsert(List<SObject> objList) {
    inserted.addAll(objList);
  }
  public void dmlUpdate(List<SObject> objList) {
    updated.addAll(objList);
  }
  // … one method per kind of DML
  public void dmlDelete(List<SObject> objList) {
    deleted.addAll(objList);
  }
}
```

The inert test

```apex
// Given: stubbed Selector and GUIDs, as before, plus
WDGT_InertDml inertDml = new WDGT_InertDml();
WDGT_UnitOfWork.mock = new fflib_SObjectUnitOfWork(
  new List<SObjectType>{ WDGT_Widget__c.SObjectType },
  inertDml
);

// When
new WDGT_WidgetsService(selectorMock, utilityMock)
  .tagWidgetsWithGuid(widgetIds);

// Then: the records that would have been saved
WDGT_Widget__c firstUpdated =
  (WDGT_Widget__c) inertDml.updated[0];
Assert.areEqual(firstGuid,
  firstUpdated.WDGT_Guid__c, '…');
Assert.isTrue(inertDml.inserted.isEmpty(), '…');
```

Real services touch many objects. The inert IDML gives inserted records fake Ids, so links resolve, and hands back one type at a time.

```apex
uow.registerNew(acme);
uow.registerNew(coyote, Contact.AccountId, acme);
uow.commitWork();
// without fake Ids, every link is null
Contact saved = (Contact) inertDml.getInserted(Contact.SObjectType)[0];
Assert.areEqual(acme.Id, saved.AccountId, '…');
```

💡 `verify` checks that a call happened; the inert IDML checks the outcome, so it doesn't care which `register*` call produced it.

Testing triggers

One save, for coverage

- **Keep the trigger a one-line handoff and the handler thin:** it converts trigger specifics to general Apex and calls other logic. Unit-test that logic where it lives.
- **Coverage only:** switch the handler off with fflib's own switch, then save one record. The trigger and the handler's construction are covered; no handler logic runs.

WDGT_Widgets.trigger

```
trigger WDGT_Widgets on WDGT_Widget__c (
    before insert, before update,
    after insert, after update) {
  fflib_SObjectDomain.triggerHandler(
    WDGT_WidgetsTriggerHandler.class
  );
}
```

WDGT_WidgetsTriggerHandlerTest.cls

```
// Given every trigger event for the handler is off
fflib_SObjectDomain
  .getTriggerEvent(WDGT_WidgetsTriggerHandler.class)
  .disableAll();

// When a widget is saved
Database.SaveResult result =
  Database.insert(new WDGT_Widget__c());

// Then it saves
Assert.isTrue(result.isSuccess(), '…');
```

⚠️ `disableAll()` mutes only our handler. **The save is real:** populate what the org requires (required fields, validation rules), and expect flows and other automation to run.

Service tests at real scale

Same patterns, bigger method

- **The Warehouse app's** `DispatchServiceTest` — Selectors in through the constructor; Domains and the Unit of Work through their static mocks (slide 14)

DispatchServiceTest.cls (inherited, trimmed)

```
// Given: three Selectors, three Domains and a Unit of Work, all mocks; records in memory, fake Ids
mocks.startStubbing();
mocks.when(lineSelectorMock.selectPendingUnassignedByWarehouseWithOrder(warehouseIds))
  .thenReturn(new List<FulfillmentLine__c>{ line });
mocks.when(robotSelectorMock.selectIdleByWarehouseWithModel(warehouseIds))
  .thenReturn(new List<Robot__c>{ robot });
mocks.when(robotsMock.getAvailableForWork())                // logic calling logic
  .thenReturn(new List<Robot__c>{ robot });
mocks.when(orderSelectorMock.selectById(new Set<Id>{ orderId }))
  .thenReturn(new List<FulfillmentOrder__c>{ order });
mocks.stopStubbing();
Robots.mock = robotsMock;
FulfillmentLines.mock = linesMock;
FulfillmentOrders.mock = ordersMock;
UnitOfWork.mock = uowMock;

// When
new DispatchService(lineSelectorMock, robotSelectorMock, orderSelectorMock)
  .dispatchWarehouses(warehouseIds);

// Then
((Robots) mocks.verify(robotsMock)).startWork(uowMock);
((FulfillmentLines) mocks.verify(linesMock)).assign(
  (Map<Id, Id>) fflib_Match.anyObject(), (fflib_SObjectUnitOfWork) fflib_Match.eq(uowMock)
);
((FulfillmentOrders) mocks.verify(ordersMock)).markInProgress(uowMock);
((fflib_SObjectUnitOfWork) mocks.verify(uowMock, 1)).commitWork();
```

💡 One behaviour per test, with a name that says what it does: `method_WHEN_condition_EXPECT_result`.

Testing a Domain

In memory; mock only what it calls out to

RobotsTest.cls (inherited)

```
// Given robots at each published due rule, plus one already in service
Robot__c intervalDue = newRobot(Robots.STATUS_IDLE, Robots.HEALTH_HEALTHY, 0, 80);
intervalDue.HoursSinceService__c = 40;
intervalDue.Model__r = new RobotModel__c(ServiceIntervalHours__c = 40);
Robot__c critical = newRobot(Robots.STATUS_IDLE, Robots.HEALTH_CRITICAL, 81, 80);
Robot__c wearDue = newRobot(Robots.STATUS_IDLE, Robots.HEALTH_WARNING, 90, 80);
Robot__c inService = newRobot(Robots.STATUS_IN_SERVICE, Robots.HEALTH_CRITICAL, 100, 80);
inService.HoursSinceService__c = 40;
Robot__c healthy = newRobot(Robots.STATUS_IDLE, Robots.HEALTH_HEALTHY, 0, 80);

// When due robots are collected
Set<Robot__c> due = Robots.newInstance(
    new List<Robot__c>{ intervalDue, critical, wearDue, inService, healthy }
  )
  .getDueForService();

// Then interval, Critical, and 90% wear are due; InService and Healthy are not
System.assertEquals(true, due.contains(intervalDue), 'Interval hours should be due');
// … three more
System.assertEquals(false, due.contains(healthy), 'Healthy idle robot should not be due');
```

A Domain's logic runs on records built in memory, usually with nothing to mock: no DML, no SOQL, no mocks.

But mock what it calls out to, the same rule as everywhere:

- **Values a test can't predict:** a Domain that needs a new GUID mocks the GUID generator, as the Widgets tests do
- **Dates:** if the method itself uses today, pass the date in through the constructor; if another class works out the next business day, mock that call. Weekends and holidays stop mattering
- **Logic outside the Domain that does a lot of work:** mock it away

Test the method. Mock the rest.

Inherited test; new tests use the `Assert` class.

What to mock at each layer

Each layer checks its own decisions

LAYER	ITS TEST CHECKS	MOCK
Controller	What it passes in, and what it shows	The Service
Service	How it orchestrates	Selectors, Domains, the Unit of Work, logic classes (`WDGT_WidgetsUtility`)
Domain	Its rules, on in-memory records	Only what it calls out to (slide 27)
Selector	That its query is valid	Nothing: empty conditions (slide 22)
Trigger	Coverage only	Nothing: the handler switched off, one save (slide 25)

💡 Each layer's test checks only its own decisions.

Pitfalls and gotchas

Learned the hard way

Test design

- **Over-verifying:** asserting every call tests the implementation, not the behaviour
- **Mocks that return mocks:** a sign the method knows too much
- **Verifying your own stub:** stub a Selector, call it, verify it. That tests nothing
- **Unrealistic stubs:** a Selector throws a `QueryException`, never a `DmlException`. A hand-made `DmlException` has no row data. Fix the stub, never the business logic

ApexMocks gotchas

- **Forgetting `stopStubbing()` :** every later call becomes a stub
- **SObject arguments match by equality:** every field. Use matchers when you can't be exact
- **Mocks of the same type share stubs** by default (`HasIndependentMocks`)
- **Matchers are all or nothing** per call; use `isNull()`, not `eq(null)`
- **One `fflib_ApexMocks` per test**
- **Arguments match by value, at call time:** List order matters, and changing an argument after stubbing or calling breaks the match

⚠️ **Tests that depend on DML break when the org changes, not the code:** a new validation rule, flow or required field. Mocked and inert tests stay green.

⚠️ **Mocked Selectors hide field gaps:** expected; integration and E2E testing catch them (slide 22).

Test the method. Mock the rest.

Promises kept

✅ `newInstance()`, not `new`	Andy, S1
✅ Instance methods, top-level classes	Andy, S1
✅ The Application class is optional	Andy, S1
✅ Mocking the Unit of Work	Andy, S2
✅ The Domain's static mock hook	John Daniel, S3
✅ Statics, and `virtual`	Session 4

What's next

Session #6 — Enterprise-Scale Apex Across Multiple Packages

John Daniel · packages and AT4DX · Nov 3

The same seams, with metadata from any package deciding what you get.

Thank you!

John Storey · Code With Sally

FFLib Series · Session 005

